



Foundations of Programming

Assignment No. : 11

PROBLEM STATEMENT:

Write a program in C to accept a number from the user and compute the following:

- a) Square root of the number
- b) Square of the number
- c) Cube of the number
- d) Check whether the number is prime
- e) Factorial of the number
- f) Prime factors of the number

OBJECTIVE:

- To understand the use of mathematical operations in C.
- To implement loops and conditional statements
- To perform number-based computations.
- To enhance logical thinking using menu-driven programs.

THEORY:

Numbers and their mathematical properties play a vital role in programming. Operations such as square, cube, and square root are basic mathematical computations. Prime number checking involves verifying whether a number has exactly two divisors. Factorial is calculated by multiplying a sequence of integers. Prime factorization breaks a number into its prime components. In C programming, these operations can be efficiently implemented using loops, conditional statements, and mathematical functions.

PLATFORM:

Linux

ALGORITHM:

Step 1: Start

Step 2: Declare variables

Integer n, i

Float sqrt

Integer fact = 1, isPrime = 1

Step 3: Accept a number n from the user.

Step 4: Compute square root $\rightarrow \text{sqrt} = \sqrt{n}$

Step 5: Compute square $\rightarrow n \times n$

Step 6: Compute cube $\rightarrow n \times n \times n$

Step 7: Check prime number

- If $n \leq 1 \rightarrow$ Not prime
- Else check divisibility from 2 to $n/2$
 - If divisible $\rightarrow$ Not prime
 - Else $\rightarrow$ Prime

Step 8: Compute factorial

fact = $1 \times 2 \times \dots \times n$

Step 9: Find prime factors

For each number from 2 to n

While divisible $\rightarrow$ print factor and divide n

Step 10: Display all results.

Step 11: Stop

C PROGRAM:

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main() {
```

```
    int n, choice, i, fact, isPrime, temp;
```

```
    char cont;
```

```
    do {
```

```
/* Display menu */  
printf("\n--- MENU ---\n");  
printf("1. Square root\n");  
printf("2. Square\n");  
printf("3. Cube\n");  
printf("4. Check Prime\n");  
printf("5. Factorial\n");  
printf("6. Prime Factors\n");  
printf("7. Exit\n");  
printf("Enter your choice: ");  
scanf("%d", &choice);  
  
if (choice >= 1 && choice <= 6) {  
    printf("Enter a number: ");  
    scanf("%d", &n);  
}  
  
switch (choice) {  
  
    case 1:  
        if (n >= 0)  
            printf("Square root = %.2f\n", sqrt(n));  
        else  
            printf("Square root not defined for negative numbers\n");  
        break;
```

case 2:

```
printf("Square = %d\n", n * n);
```

```
break;
```

case 3:

```
printf("Cube = %d\n", n * n * n);
```

```
break;
```

case 4:

```
isPrime = 1;
```

```
if (n <= 1)
```

```
    isPrime = 0;
```

```
else {
```

```
    for (i = 2; i <= n / 2; i++) {
```

```
        if (n % i == 0) {
```

```
            isPrime = 0;
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
if (isPrime)
```

```
    printf("Number is Prime\n");
```

```
else
```

```
    printf("Number is Not Prime\n");
```

```
break;
```

case 5:

```
if (n < 0) {  
    printf("Factorial not defined for negative numbers\n");  
} else {  
    fact = 1;  
    for (i = 1; i <= n; i++)  
        fact *= i;  
    printf("Factorial = %d\n", fact);  
}  
break;
```

case 6:

```
printf("Prime factors: ");  
temp = n;  
for (i = 2; i <= temp; i++) {  
    while (temp % i == 0) {  
        printf("%d ", i);  
        temp /= i;  
    }  
}  
printf("\n");  
break;
```

case 7:

```
printf("Exiting program.\n");
```

```
break;
```

```
default:
```

```
printf("Invalid choice! Try again.\n");
```

```
}
```

```
if (choice != 7) {
```

```
printf("\nDo you want to perform another operation? (y/n): ");
```

```
scanf(" %c", &cont);
```

```
} else {
```

```
cont = 'n';
```

```
}
```

```
} while (cont == 'y' || cont == 'Y');
```

```
return 0;
```

```
}
```

```
1  #include <stdio.h>
2  #include <math.h>
3
4  int main() {
5      int n, choice, i, fact, isPrime, temp;
6      char cont;
7
8      do {
9          /* Display menu */
10         printf("\n--- MENU ---\n");
11         printf("1. Square root\n");
12         printf("2. Square\n");
13         printf("3. Cube\n");
14         printf("4. Check Prime\n");
15         printf("5. Factorial\n");
16         printf("6. Prime Factors\n");
17         printf("7. Exit\n");
18         printf("Enter your choice: ");
19         scanf("%d", &choice);
20
21         if (choice >= 1 && choice <= 6) {
22             printf("Enter a number: ");
23             scanf("%d", &n);
24         }
25
26         switch (choice) {
27
28             case 1:
29                 if (n >= 0)
```

```

30         printf("Square root = %.2f\n", sqrt(n));
31     else
32         printf("Square root not defined for negative numbers\n");
33     break;
34
35     case 2:
36         printf("Square = %d\n", n * n);
37         break;
38
39     case 3:
40         printf("Cube = %d\n", n * n * n);
41         break;
42
43     case 4:
44         isPrime = 1;
45         if (n <= 1)
46             isPrime = 0;
47         else {
48             for (i = 2; i <= n / 2; i++) {
49                 if (n % i == 0) {
50                     isPrime = 0;
51                     break;
52                 }
53             }
54         }
55

```

```

56     if (isPrime)
57         printf("Number is Prime\n");
58     else
59         printf("Number is Not Prime\n");
60     break;
61
62     case 5:
63         if (n < 0) {
64             printf("Factorial not defined for negative numbers\n");
65         } else {
66             fact = 1;
67             for (i = 1; i <= n; i++)
68                 fact *= i;
69             printf("Factorial = %d\n", fact);
70         }
71         break;
72
73     case 6:
74         printf("Prime factors: ");
75         temp = n;
76         for (i = 2; i <= temp; i++) {
77             while (temp % i == 0) {
78                 printf("%d ", i);
79                 temp /= i;
80             }
81         }

```



```

80         }
81     }
82     printf("\n");
83     break;
84
85     case 7:
86         printf("Exiting program.\n");
87         break;
88
89     default:
90         printf("Invalid choice! Try again.\n");
91     }
92
93     if (choice != 7) {
94         printf("\nDo you want to perform another operation? (y/n): ");
95         scanf(" %c", &cont);
96     } else {
97         cont = 'n';
98     }
99
100 } while (cont == 'y' || cont == 'Y');
101
102 return 0;
103 }

```

INPUT:

The user enters an integer number and selects an operation from the menu.

Example Input: Number: 5 Choice: 3

OUTPUT:

Example Output:

```
--- MENU ---
1. Square root
2. Square
3. Cube
4. Check Prime
5. Factorial
6. Prime Factors
7. Exit
Enter your choice: 3
Enter a number: 5
Cube = 125
```

CONCLUSION:

Thus, the program to perform various mathematical operations on a number was successfully implemented using C programming. This program demonstrates the use of loops, conditional statements, switch-case, and mathematical functions.

FAQs:

1. Why is checking divisibility only up to $n/2$ sufficient in prime number verification, and how can this be optimized further?
2. What are the limitations of using int data type for factorial calculation, and how can overflow be handled in C?
3. How does the time complexity of prime factorization change with increasing input size?
4. Why is the math.h library required for square root calculation, and what happens if it is not included?
5. How would the program behaviour change if a negative number is entered for factorial or square root operations?
6. Explain the difference between checking a number for primality and generating prime factors of the same number.
7. How can recursion be used to calculate factorial instead of loops, and what are its advantages and disadvantages?